MODULE 14 · DOMAIN 5 - TERRAFORM MODULES

Modules: Sourcing, Scope, Composition

Where a module comes from, what crosses its boundary, and when not to write one.

DOMAIN 5 of 8 COURSE WEIGHT 10% MODULE 14 of 27

admin@sthithapragna.com

A module is **a directory containing Terraform files**. Nothing more. You have been writing one all along.

Root and child

- The **root module** is the directory where you run Terraform. Every configuration has exactly one.
- A **child module** is any module called by another, through a **module** block.
- Child modules can call further children. The relationships form a tree with the root at the top.
- **Terraform reads only the current directory.** Subdirectories are separate modules, not part of this one.
- So a module is **not** a special file or a keyword. It is a directory, viewed from outside.

> ON THE EXAM
>
> Module eight's point again: Terraform reads every .tf file in one directory and no further. That is what makes a directory a module.

The module block

- **module**, then a name you choose, which becomes part of the address for everything inside.
- **source** is the only required argument. It tells Terraform where to find the code.
- **version** applies to registry sources only. Module fifteen covers it.
- **Every other argument sets one of the module's input variables.** There is no separate inputs block.
- **Changing source requires terraform init again.** Module installation is one of init's three jobs.

> ON THE EXAM
>
> count, for_each, depends_on and providers are also available on a module block. Everything else is an input variable.

Where a module can come from

SOURCE TYPE	NOTES
Local paths	Must begin ./ or ../ . Not downloaded, read in place
Terraform Registry	NAMESPACE/NAME/PROVIDER. Supports the version argument
Private registry	The same, with a hostname in front
GitHub and Bitbucket	Recognised shorthand, no prefix needed
Generic Git and Mercurial	git:: or hg:: prefix, with a ref argument
HTTP URLs, S3 and GCS buckets	Archive files fetched over HTTP or from object storage

ON THE EXAM

Only registry sources support the version argument. Every other source pins by ref, by URL, or not at all.

Local paths

- **A local path must begin with ./ or ../**, which is how Terraform tells it apart from a registry address.
- Write a bare directory name and Terraform will try the **public registry** instead, and fail confusingly.
- **Local paths are not installed.** The files are already on disk, so Terraform just reads them where they are.
- So **no version argument**, and no copy in the modules cache. Whatever is on disk is what runs.
- Edit a local module and the change is live on the next plan, which makes development fast.

> ON THE EXAM
>
> The leading dot-slash is not decoration. Without it the string is a registry address, and that is a whole different lookup.

An absolute path is not a local path

- **Terraform does not treat an absolute filesystem path as a local path.** Not a leading slash, not a drive letter.
- It treats it like a **remote module** and copies it into the local module cache.
- So edits stop taking effect immediately, and **init must be re-run** to pick them up.
- HashiCorp recommends against absolute paths, because they **couple your configuration to one machine's layout**.
- Relative paths are portable. Absolute ones work on your laptop and nowhere else.

> ON THE EXAM
>
> Two things that look like local paths, and only one behaves like one. The relative prefix is what triggers the local behaviour.

Registry and Git sources

- **Public registry:** three parts, namespace, name and target provider, separated by slashes.
- **Private registry:** the same three parts with a hostname in front, making four.
- **GitHub and Bitbucket are detected automatically** from the address. No prefix required.
- **Generic Git needs the git:: prefix**, and should always carry a **ref** argument.
- **Without ref, you get the default branch, whatever it happens to contain today.**

> ON THE EXAM
>
> A Git source with no ref is the module equivalent of a provider with no version constraint. It works until somebody merges something.

Sub-directories, with a double slash

- When a source is a repository or archive, the module may live in a **sub-directory** of it.
- A **double slash** marks where the package ends and the path within it begins.
- **The sub-directory portion must come before any arguments**, such as ref. Order matters.
- Terraform still downloads the whole package, and then reads only that sub-directory.
- HashiCorp recommends putting each reusable module in the **root of its own repository** where possible.

> ON THE EXAM
>
> This is how a monorepo of modules is consumed. One repository, many modules, each addressed after the double slash.

Reading a source address

SOURCE =	WHAT TERRAFORM DOES
`./modules/vpc`	Reads the directory in place. No install
`modules/vpc`	Looks on the public registry, and fails
`/opt/shared/vpc`	Treated as remote. Copied to the module cache
`terraform-aws-modules/vpc/aws`	Public registry. Accepts a version argument
`app.terraform.io/acme/vpc/aws`	Private registry. Also accepts version
`git::https://git.acme.io/tf.git//vpc?ref=v2.1`	Git, sub-directory vpc, pinned to tag v2.1

ON THE EXAM

The last row shows the full ordering: package, then double slash and path, then the ref argument.

HashiCorp's guidance on choosing

- **Local file paths** for closely related modules, used mainly to factor out repeated code in one repository.
- **A native module registry** for modules intended to be shared by multiple calling configurations.
- Other sources exist so you can distribute modules using **infrastructure you already have**.
- Local paths across repository boundaries are fragile, because they depend on where somebody cloned things.
- The deciding question: **is this shared beyond this repository?**

> ON THE EXAM
>
> Factoring out repetition within one repository, or distributing to other teams. Those are different problems with different answers.

Interacting with inputs and outputs

- A caller sets inputs as **arguments in the module block**, matching the child's variable names.
- A caller reads outputs as **module, the module name, then the output name**.
- **Outputs chain.** One module's output becomes another module's input, which is how flat composition works.
- **A missing required input is an error at plan time**, naming the module and the variable.
- A module's **variable descriptions and output descriptions** are its documentation. The registry publishes them.

> ON THE EXAM
>
> Chaining outputs into inputs creates the dependency between two modules, exactly as a resource reference does.

Scope: what crosses the boundary

- **Nothing is shared implicitly.** A module cannot see the caller's variables, locals or resources.
- **In: module block arguments**, which set the child's input variables. That is the only route.
- **Out: declared outputs.** Undeclared internals are unreachable, as module eight established.
- **Provider configurations are the exception.** Default ones are inherited automatically.
- **Aliased provider configurations are not inherited** and must be passed with the providers meta-argument.

> ON THE EXAM
>
> One exception to 'nothing is shared', and it is providers. That asymmetry is examinable and easy to forget.

Composition

- **Flat composition:** a root module calls several children and wires them together.
- **Deep nesting** makes values travel through layers of pass-through variables.
- Each layer must **declare, accept and forward** every value.
- Flatter trees are easier to read and produce shorter resource addresses.
- **Deep nesting also makes plan output harder to read**, as addresses grow.

> ON THE EXAM
>
> The cost of nesting is not performance. It is that every layer becomes a place where a value must be declared and forwarded.

The standard module structure

- **main.tf, variables.tf, outputs.tf** at the root of the module. Convention, not requirement.
- A **README** describing what it does, and a **LICENSE** if it is published.
- An **examples** directory showing how to call it. These are separate root modules, not part of the module.
- A **modules** subdirectory for nested modules the main one uses.
- **The registry reads this structure** to generate documentation automatically.

> ON THE EXAM
>
> Filenames still carry no meaning to Terraform. They carry meaning to the registry, to tooling, and to the next person.

When not to write a module

- **A thin wrapper around a single resource** adds a layer and removes nothing.
- If every variable is passed straight through, the module is **a rename, not an abstraction**.
- It also **constrains callers**, who can now only use the arguments you chose to expose.
- A module earns its place when it **encapsulates a decision**: several resources wired together correctly.
- The test: does calling this module save the caller from knowing something they would otherwise need to know?

> ON THE EXAM
>
> Modules are for encapsulating decisions, not for hiding resource types. A wrapper with no opinion is overhead with a version number.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Edits to the module are not taking effect	An absolute path, cached like a remote module
Terraform looked for it on the registry	A local path missing its ./ prefix
Shared across several repositories	A registry, not a relative path
Different results on different machines	A Git source with no ref argument
The module lives in a monorepo folder	A double slash before the ref argument
Every variable is passed straight through	A thin wrapper. Probably should not be a module

ON THE EXAM

Most sourcing questions are decided by one character: the leading dot, the double slash, or the missing ref.

> A developer is building a new networking module. Her root configuration references it with a source of /home/dev/work/modules/networking, an absolute path on her machine.
>
> She edits the module, reruns plan, and sees no change. Running init again picks the edit up. She expected local modules to update immediately.

What is happening?

A. Local modules are cached in state and require -refresh to update

B. An absolute path is not treated as a local path. Terraform copies it into the module cache

C. The module needs a version argument before edits are detected

D. Plan does not read modules. Only apply reinstalls them

◆ PAUSE AND DECIDE

What is happening?

A Cached in state

Modules are installed into the .terraform directory, not state. State records infrastructure bindings, as module four established.

B Absolute is not local

Correct. HashiCorp documents that an absolute path is treated similarly to a remote module and copied into the local module cache.

C Needs a version

The version argument applies to registry sources only, and would be rejected here. It has nothing to do with detecting edits.

D Only apply reinstalls

Neither plan nor apply installs modules. Installation is one of init's three jobs, which is exactly why init fixed it.

THE SIGNAL

Change the source to a relative path and the fast edit-and-plan loop returns. One character of prefix decides the behaviour.

> Nine teams across four repositories all deploy a similar service. Each repository has copied a set of module directories and references them with relative paths such as ../../shared/service.
>
> Fixes are applied inconsistently, and two repositories have diverged. The platform team wants one authoritative copy with controlled adoption.

What should they do?

A. Move the modules to one repository and reference them by absolute path from each consumer

B. Publish the modules to a private registry and reference them with a version constraint

C. Keep relative paths but add a script that copies the canonical module into each repository

D. Combine all four repositories into a monorepo so relative paths resolve

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should they do?

A Absolute paths

Requires every machine and every runner to have the same filesystem layout, which is precisely what HashiCorp warns against.

B A private registry

Correct. This is the documented purpose of a registry: modules shared by multiple calling configurations, with versioning and controlled adoption.

C A copying script

Automates the divergence rather than removing it. Every repository still holds its own copy, now with a script to maintain as well.

D Merge into a monorepo

Restructures four teams' repositories to solve a distribution problem. It also removes their ability to adopt changes independently.

THE SIGNAL

The phrase 'controlled adoption' is the tell. Only a versioned source lets a team choose when to take a change.

> A configuration sources a module from a generic Git repository with a git:: prefix and no ref argument. It has worked for a year.
>
> A pipeline run in a fresh working directory now produces a plan proposing changes nobody made. The same configuration on an engineer's laptop, initialised months ago, plans no changes.

What is the cause, and the fix?

A The dependency lock file was not committed, so module versions diverged

B With no ref, each init takes the default branch as it is then. Pin with a ref to a tag

C The laptop's state is stale and should be refreshed

D Git sources require terraform init -upgrade on every run to stay consistent

◆ PAUSE AND DECIDE

What is the cause, and the fix?

A Missing lock file

The lock file tracks providers only, as module three established. It has never recorded module versions, so committing it would change nothing here.

B No ref, so the branch moved

Correct. Without a ref, init takes the default branch as it stands at that moment. The two environments installed at different times.

C Stale state on the laptop

Both environments share the same remote state. What differs is the module code each has installed, which state knows nothing about.

D Always use -upgrade

Would guarantee everyone tracks the moving branch, making the problem universal rather than intermittent.

THE SIGNAL

Module three's finding, arriving as an incident. Providers are locked and modules never were, so an unpinned module is genuinely unpinned.

> A team has written a module containing exactly one storage bucket resource. It declares eleven input variables, each passed straight through to the corresponding resource argument, and one output returning the bucket name.
>
> A reviewer is asked whether this should be published to the private registry.

What is the strongest response?

A. Yes. Every resource should be wrapped in a module for consistency

B. Probably not. It adds a layer and a release cycle while encapsulating no decision

C. Yes, but only after adding count so it can be made conditional

D. No. Modules containing a single resource are rejected by the registry

◆ PAUSE AND DECIDE

What is the strongest response?

A Wrap everything

Consistency for its own sake. It also restricts callers to the eleven arguments chosen, where the resource offers many more.

B It encapsulates nothing

Correct. Eleven pass-through variables is a rename, not an abstraction. A module should save the caller from knowing something.

C Add count first

Improves a module that should not exist. Conditional creation works equally well on the resource directly.

D The registry rejects it

Invents a rule. Single-resource modules are perfectly valid and sometimes correct, when they encode defaults or policy.

THE SIGNAL

A single-resource module is fine when it encodes an opinion: enforced encryption, naming, logging. Pure pass-through encodes nothing.

Module 14 recap

- **A module is a directory of Terraform files.** The root module is one, and every configuration has it.
- **Local paths must begin ./ or ../**, and are read in place rather than installed.
- **An absolute path is not a local path.** It is cached like a remote module.
- **Local paths for factoring out repetition; a registry for sharing across configurations.**
- **Nothing crosses the boundary except inputs and outputs**, apart from default provider configurations.
- **A Git source with no ref is unpinned**, and the lock file will not save you.

> ON THE EXAM
>
> A module should encapsulate a decision. Eleven pass-through variables around one resource encapsulates nothing.